

An Industrial Oriented Mini Project On
VOICE CONTROLLER DESKTOP ASSISTANCE

Submitted to partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

DEEPA SRI VADLA

[23911A0514]

Under the Guidance of

Dr. Md Zaheer Ahmed

Assistant Professor



Department of Computer Science and Engineering

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY
(An Autonomous Institution)

(Approved by AICTE, accredited by NAAC, NBA C permanently Affiliated to JNTUH, Hyderabad)

Aziz Nagar Gate, C.B. Post, Hyderabad-500075

2025-2026



(Approved by AICTE, accredited by NAAC, NBA C permanently Affiliated to JNTUH, Hyderabad)

Aziz Nagar Gate, C.B. Post, Hyderabad-500075

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project titled “**VOICE CONTROLLER DESKTOP ASSISTANCE**” is submitted by Deepa Sri Vadla (23911A0514) in partial fulfilment for the award of the degree of Bachelor of Technology in **Computer Science and Engineering**, is a record of bonafide work carried out by her under my guidance and supervision. These results embodied in this project report have not been submitted to any other University or Institute for the award of any degree.

Internal Guide

Dr. Md Zaheer Ahmed

Assistant Professor

Head of Department

Dr. D. Aruna Kumari

Professor

DECLARATION

I, Deepa Sri Vadla hereby declare that the project entitled, “**VOICE CONTROLLER DESKTOP ASSISTANCE**” submitted for the award of Bachelor of Technology in Computer Science and Engineering is original and has been done by me and this work is not copied and submitted anywhere for the award of any degree.

DEEPA SRI VADLA

[23911A0514]

ACKNOWLEDGEMENT

I wish to express our sincere gratitude to the project guide, **Dr. Md Zaheer Ahmed**, Assistant Professor, Vidya Jyothi Institute of Technology, Hyderabad for his timely cooperation and valuable suggestions while carrying out this work. It is his kindness that made us learn more from him

I am grateful to **Dr. D Aruna Kumari**, Professor, and HOD, department of CSE, for her help and support during our academic year.

I am wholeheartedly convey my gratitude to the Principal **Dr. A. Srujana**, for providing the necessary infrastructure to complete this project.

I would thank my parents and all the faculty members who have contributed to my progress through the course to come to this stage.

Deepa Sri Vadla

[23911A0514]

ABSTRACT

VOICE CONTROLLER DESKTOP ASSISTANCE

Jarvis is a Python-based voice assistant designed to enable users to interact with computers using voice commands instead of traditional input methods like keyboard and mouse. The system captures audio input through a microphone, converts speech into text using speech recognition techniques, processes the command using Python logic, and generates a response through text-to-speech conversion. The assistant is capable of performing various tasks such as opening websites, searching information from Wikipedia, playing music, and providing real-time information like current time.

The system is developed using Python libraries such as SpeechRecognition for voice input processing and pyttsx3 for speech output generation. The main objective of this project is to create a simple, efficient, and user-friendly voice assistant that demonstrates the basic concepts of automation and human-computer interaction. This project provides practical understanding of speech processing systems and shows how intelligent assistants can simplify daily computer operations.

INDEX

S.NO	TITLE	PAGE NO.
1	INTRODUCTION	1
2	LITERATURE SURVEY	2
3	FEASIBILITY STUDY	3
	3.1 Technical Feasibility	3
	3.2 Economic Feasibility	3
	3.3 Operational Feasibility	3
4	SYSTEM REQUIREMENTS	4
	4.1 Existing System	4
	4.2 Proposed System	4
	4.3 Hardware Requirements	4
	4.4 Software Requirements	4
5	SYSTEM DESIGN	5-6
	5.1 Design Objectives	5
	5.2 Data Flow Diagram (Dfd)	5
	5.2.1 Data Flow Diagram	5
	5.3 System Architecture	6
6	SYSTEM IMPLEMENTAION	7-8

6.1 Voice Input Module	7
6.2speech Recognition Module	7
6.3 Command Processing Module	7
6.4 Task Execution Module	7
6.5 Text-Tospeech Module	8
7 SAMPLE CODE	9-13
7.1 Assistant.Py	9
7.2 Commands.Py	9
7.3 Config.Py	9
7.4 Jarvis_App.Py	10
7.5 Listen.Py	11
7.6 Mic_Test.Py	12
7.7 Speak.Py	13
8 SYSTEM TESTING	14- 19
8.1 Unit Testing	14
8.2 Integration Testing	14
8.3 User Acceptance Testing (Uat)	15
8.4 Output Testing	15
9 RESULT & OUTPUT	16-21

9.1 Voice Input Recognition	16
9.2 Opening Applicationa	16
9.3 Wikipedia Search	16
9.4 Music Playback	17
9.5 Time Announcement	17
9.6 Email Sending	17
9.7 Voice Response Output	17
9.8 Overall Performace	18
9.9 Sample Output Screens	18
10 CONCLUSION	22
11 FUTURE SCOPE	23-24
11.1 Integration With Ai	23
11.2 Multi – Languge Support	23
11.3 Mobile Application Development	23
11.4 Integration With Smart Devices	23
11.5 Enhanced Security Features	24
11.6 Advance Features And Automation	24
12 REFERENCES	25

CHAPTER 1

INTRODUCTION

In recent years, voice-based technologies have gained significant importance in improving the way users interact with computers. Voice assistants allow users to perform tasks through speech, eliminating the need for manual input devices. This not only saves time but also provides convenience, especially for users who prefer hands-free operation.

This project focuses on the development of a voice assistant named Jarvis using Python. The system is designed to understand user commands through voice input and perform appropriate actions such as opening applications, retrieving information, and responding through speech output. The assistant uses speech recognition to convert audio into text and text-to-speech technology to generate responses.

The primary goal of this project is to develop a simple and efficient voice assistant that demonstrates automation and improves human–computer interaction. The project also helps in understanding real-time processing, command handling, and integration of multiple Python libraries to build an intelligent system.

CHAPTER 2

LITERATURE SURVEY

Voice assistants such as Apple Siri, Amazon Alexa, and Google Assistant are widely used in modern systems. These assistants use advanced technologies like Natural Language Processing (NLP), machine learning, and cloud computing to provide accurate and intelligent responses.

Research studies indicate that speech recognition systems have significantly improved user experience by enabling faster and more natural interaction. Many applications use APIs and cloud-based services for processing voice commands and generating responses.

Python has become a popular language for developing such systems due to its simplicity and availability of powerful libraries like SpeechRecognition and pyttsx3. These libraries allow developers to build basic voice assistants without requiring complex infrastructure. This project uses these technologies to implement a simple desktop-based assistant for educational and practical purposes.

CHAPTER 3

FEASIBILITY STUDY

Feasibility study is conducted to evaluate the practicality and effectiveness of the Jarvis voice assistant system. It helps in understanding whether the project can be successfully implemented with available resources. The feasibility of the system is analyzed under three main categories: technical, economic, and operational feasibility.

3.1 Technical Feasibility

The Jarvis voice assistant is developed using Python, which is a widely used and easy-to-learn programming language. The system uses libraries such as SpeechRecognition and pyttsx3 for speech processing, which are readily available and easy to integrate. The project does not require complex hardware or high-end systems. It can run on a standard computer with a microphone and internet connection. The technologies used are reliable and well-supported, making the system technically feasible.

3.2 Economic Feasibility

The project is cost-effective as it uses open-source software and libraries. There is no need for expensive tools or licenses. Development can be carried out using a basic computer system, which reduces the overall cost. Since the system does not require additional hardware or paid services, it is economically feasible for students and small-scale applications.

3.3 Operational Feasibility

The system is designed to be simple and user-friendly. Users can interact with the assistant using voice commands without needing technical knowledge. The interface is easy to use, and the system provides clear voice responses. The Jarvis assistant reduces manual effort and improves efficiency, making it suitable for everyday use. Therefore, the system is operationally feasible.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 Existing System

- Relies on keyboard and mouse input
- Requires manual effort
- Time-consuming for repeated tasks

4.2 Proposed System

- Uses voice commands for interaction
- Faster and more efficient
- Reduces manual effort
- Improves user experience

4.3 Hardware Requirements

- Computer or Laptop
- Microphone for voice input
- Speaker for audio output

4.4 Software Requirements

- Python Programming Language
- SpeechRecognition Library
- pyttsx3 Library
- Wikipedia Library
- IDE (VS Code / PyCharm)

CHAPTER 5

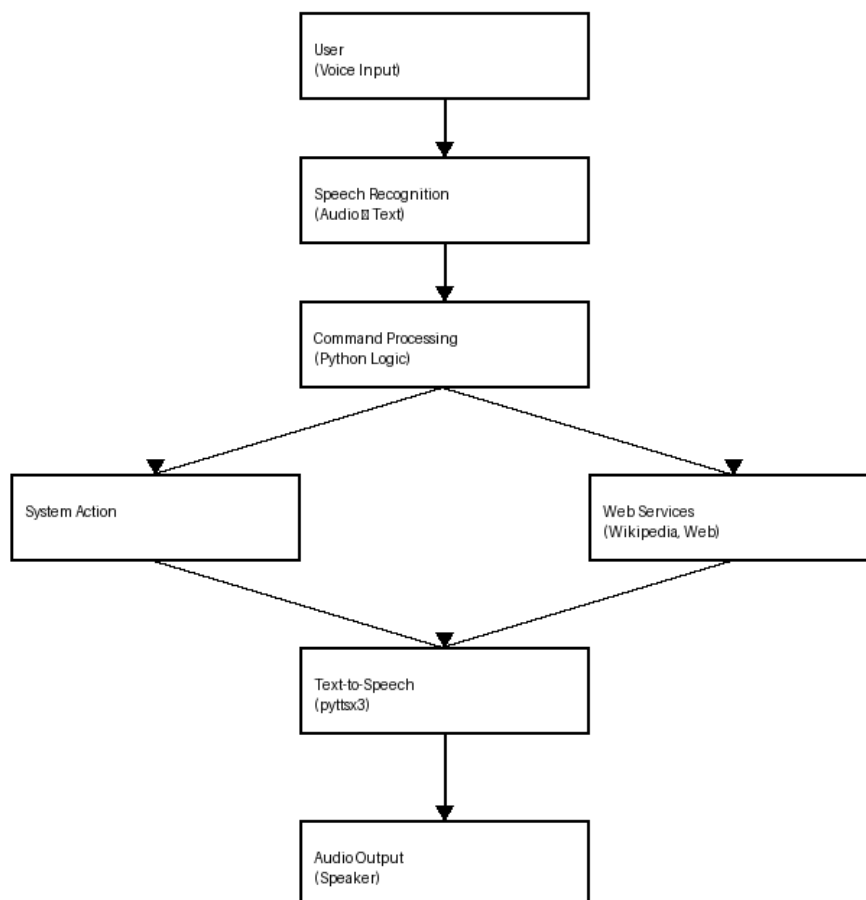
SYSTEM DESIGN

5.1 Design Objectives

- To create a simple and efficient voice assistant
- To enable speech-based interaction
- To automate basic system tasks
- To ensure quick and accurate responses

5.2 Data Flow Diagram

The user provides voice input which is converted into text using speech recognition. The command is processed using Python logic, and the system performs the required action either locally or through web services. Finally, the output is converted into speech and delivered to the user.



5.2.1 Data Flow Diagram

5.3 System Architecture

The system architecture consists of several components working together. The user provides voice input through a microphone, which is processed by the SpeechRecognition module to convert speech into text. The text is then analyzed by the command processing unit using Python logic. Based on the command, the system performs actions such as opening applications or retrieving information. Finally, the response is converted into speech using the pyttsx3 module and delivered to the user through speakers.

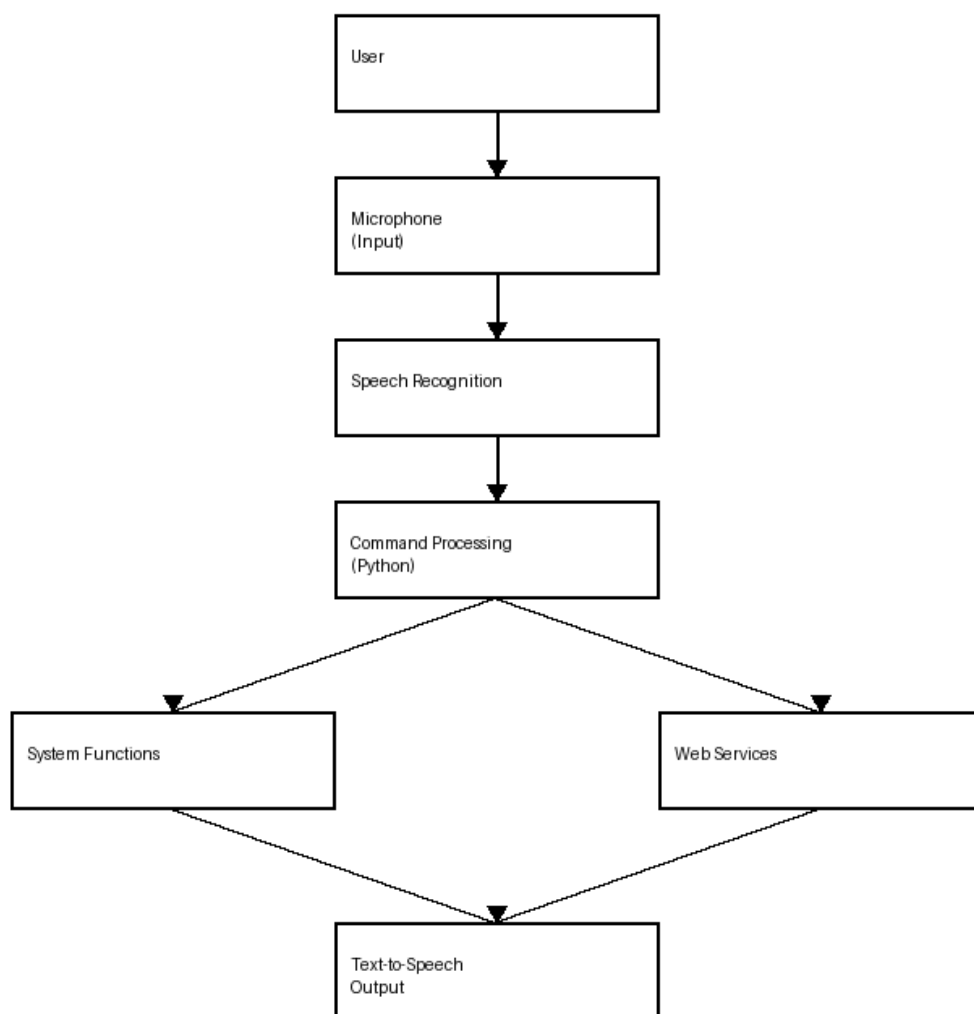


Fig 5.3.1 System Architecture

CHAPTER 6

SYSTEM IMPLEMENTATION

The Jarvis voice assistant is implemented using Python programming language with the help of various libraries that enable speech recognition, text-to-speech conversion, and system automation. The system is designed in a modular way, where each component performs a specific task, ensuring simplicity and easy maintenance.

6.1 Voice Input Module

The voice input module is responsible for capturing the user's voice through the microphone. The SpeechRecognition library is used to record audio input. The system continuously listens for user commands and captures the audio data for further processing.

6.2 Speech Recognition Module

This module converts the captured audio input into text using the Google Speech Recognition API. The recognized text is then passed to the command processing unit. Error handling is included to manage cases where the input is unclear or not recognized.

6.3 Command Processing Module

The command processing module is the core part of the system. It analyzes the text input using Python conditional statements (if-elif) and determines the appropriate action to be performed. Based on the user's query, it selects the corresponding function or operation.

6.4 Task Execution Module

This module performs the required operations based on the processed command. It uses different Python libraries such as:

- **webbrowser** to open websites
- **os** to access system files and applications
- **datetime** to provide time-related responses

6.5 Text-to-Speech Module

The response generated by the system is converted into speech using the pyttsx3 library. This module ensures that the assistant communicates with the user in a natural and understandable way.

6.6 Email Module

The system includes an optional email feature using the smtplib library. It allows users to send emails through voice commands. The module connects to an SMTP server, authenticates the user, and sends the email content.

6.7 System Workflow

The overall workflow of the system is as follows:

1. User gives voice input
2. Speech is converted into text
3. Command is processed using Python logic
4. Required action is executed
5. Output is converted into speech and delivered to the user

6.8 Technologies Used

- **Python** – Core programming language
- **SpeechRecognition** – Voice input processing
- **pyttsx3** – Text-to-speech conversion
- **Wikipedia API** – Information retrieval
- **OS & Webbrowser modules** – System control

CHAPTER 7

SAMPLE CODE

Assistant.py

```
from listen import listen
from speak import speak
from commands import process_command

WAKE_WORDS = ["hey jarvis", "jarvis"]

def has_wake_word(text):
    for word in WAKE_WORDS:
        if word in text:
            return True
    return False

if __name__ == "__main__":
    speak("Hello! Say 'Hey Jarvis' to wake me up.")

    while True:
        print("\nWaiting for wake word...")
        text = listen()

        if text == "none":
            continue

        if has_wake_word(text):
            speak("I'm listening...")
            command = listen()

            if command != "none":
                process_command(command)
```

Commands.py

```
import os
from speak import speak

def process_command(command):
    command = command.lower().strip()

    # ----- OPEN COMMANDS -----
    if "open notepad" in command:
        speak("Opening Notepad")
        os.system("start notepad")

    elif "open chrome" in command:
        speak("Opening Chrome")
        os.system("start chrome")

    elif "open calculator" in command or "open calc" in command:
        speak("Opening Calculator")
        os.system("start calc")

    # ----- CLOSE COMMANDS -----
    elif "close notepad" in command:
        speak("Closing Notepad")
        os.system("taskkill /f /im notepad.exe")
```

```

elif "close chrome" in command:
    speak("Closing Chrome")
    os.system("taskkill /f /im chrome.exe /t")

elif "close calculator" in command or "close calc" in command:
    speak("Closing Calculator")
    os.system("taskkill /f /im calculator.exe")

# ----- EXIT -----
elif "stop jarvis" in command or "exit" in command:
    speak("Goodbye")
    exit()

else:
    speak("I did not understand the command")

```

Config.py

```

APP_PATHS = {
    "chrome": r"C:\Program Files\Google\Chrome\Application\chrome.exe",
    "notepad": "notepad.exe",
    "music_folder": r"F:\Music"    # change to your music folder
}

DEFAULT_CITY = "Hyderabad"

# Add your API keys
OPENAI_API_KEY = "YOUR_OPENAI_API_KEY"
NEWS_API = "YOUR_NEWS_API_KEY"

```

Jarvis_app.py

```

import tkinter as tk
import threading
from speak import speak
from listen import listen
from commands import process_command

WAKE_WORDS = ["hey jarvis", "hello jarvis", "jarvis"]
listening = False

def set_mic(state):
    if state:
        mic_label.config(text="🎧 Listening", fg="green")
    else:
        mic_label.config(text="🔴 Not Listening", fg="red")

def jarvis_loop():
    global listening
    speak("Jarvis activated")

    while listening:
        status.config(text="Listening for wake word...")
        text = listen()

        if not listening:
            break

        if text == "none":
            continue

        if any(w in text for w in WAKE_WORDS):
            speak("Yes")
            status.config(text="Listening to command...")

            command = listen()

```

```

if command != "none":
    status.config(text="Processing...")
    process_command(command)
    status.config(text="Listening...")

    set_mic(False)
    status.config(text="Stopped")

def start_listening():
    global listening
    if listening:
        return
    listening = True
    set_mic(True)
    threading.Thread(target=jarvis_loop, daemon=True).start()

def stop_listening():
    global listening
    listening = False
    set_mic(False)
    speak("Jarvis stopped")

# ----- UI -----
window = tk.Tk()
window.title("Jarvis Desktop Assistant")
window.geometry("380x300")
window.resizable(False, False)

title = tk.Label(window, text="JARVIS", font=("Arial", 22, "bold"))
title.pack(pady=15)

mic_label = tk.Label(window, text="🔊 Not Listening", font=("Arial", 12))
mic_label.pack(pady=5)

btn_start = tk.Button(
    window,
    text="🔊 Ask Jarvis (Start)",
    font=("Arial", 13),
    width=22,
    command=start_listening
)
btn_start.pack(pady=10)

btn_stop = tk.Button(
    window,
    text="🔇 Stop Listening",
    font=("Arial", 12),
    width=22,
    command=stop_listening
)
btn_stop.pack(pady=5)

status = tk.Label(window, text="Idle", font=("Arial", 10))
status.pack(pady=15)

window.mainloop()

Listen.py

import sounddevice as sd
import speech_recognition as sr
import tempfile
import os
import wave

def listen():
    r = sr.Recognizer()

```

```

fs = 16000          # Sample rate
duration = 5        # seconds
print("Listening...")

try:
    recording = sd.rec(
        int(duration * fs),
        samplerate=fs,
        channels=1,
        dtype='int16'
    )
    sd.wait()

    # Create temp file path (not opened)
    temp_path = tempfile.mktemp(suffix=".wav")

    # Write proper PCM WAV
    with wave.open(temp_path, 'wb') as wf:
        wf.setnchannels(1)
        wf.setsampwidth(2)
        wf.setframerate(fs)
        wf.writeframes(recording.tobytes())

    # Read audio
    with sr.AudioFile(temp_path) as source:
        audio = r.record(source)

    os.remove(temp_path)

    text = r.recognize_google(audio).lower()
    print("You:", text)
    return text

except Exception as e:
    print("Listening error:", e)
    return "none"

```

Listen.py

```

import sounddevice as sd
import speech_recognition as sr
import tempfile
import os
import wave

def listen():
    r = sr.Recognizer()

    fs = 16000          # Sample rate
    duration = 5        # seconds

    print("Listening...")

    try:
        recording = sd.rec(
            int(duration * fs),
            samplerate=fs,
            channels=1,
            dtype='int16'
        )
        sd.wait()

        # Create temp file path (not opened)
        temp_path = tempfile.mktemp(suffix=".wav")

```

```

# Write proper PCM WAV
with wave.open(temp_path, 'wb') as wf:
    wf.setnchannels(1)
    wf.setsampwidth(2)
    wf.setframerate(fs)
    wf.writeframes(recording.tobytes())

# Read audio
with sr.AudioFile(temp_path) as source:
    audio = r.record(source)

os.remove(temp_path)

text = r.recognize_google(audio).lower()
print("You:", text)
return text

except Exception as e:
    print("Listening error:", e)
    return "none"

```

Mic_test.py

```

from listen import listen

print("Say something...")
text = listen()
print("Result:", text)

```

Speak.py

```

import pyttsx3

def speak(text):
    print("Assistant:", text)

    engine = pyttsx3.init()

    # --- FIX: choose working voice engine for Windows ---
    engine.setProperty('rate', 180) # speech speed
    voices = engine.getProperty('voices')

    # try different voices (0, 1, 2,...)
    try:
        engine.setProperty('voice', voices[1].id) # male voice usually index 1
    except:
        engine.setProperty('voice', voices[0].id) # fallback

    engine.say(text)
    engine.runAndWait()

```

CHAPTER 8

SYSTEM TESTING

System testing is an important phase in the development of the Jarvis voice assistant. It ensures that all components of the system work correctly, efficiently, and as expected. Different types of testing were performed to verify the functionality, reliability, and performance of the system.

8.1 Unit Testing

Unit testing involves testing individual modules separately to ensure that each component works correctly.

- **Voice Input Module:**
Verified that the microphone correctly captures user voice input.
- **Speech Recognition Module:**
Tested conversion of speech to text for different voice commands and checked accuracy.
- **Command Processing Module:**
Verified that the system correctly identifies commands using conditional statements.
- **Text-to-Speech Module:**
Checked whether the system produces correct and clear voice output.

8.2 Integration Testing

Integration testing ensures that all modules work together properly.

- Verified the flow from voice input → text conversion → command processing → execution → speech output
- Checked interaction between SpeechRecognition and Python logic
- Ensured smooth communication between system modules without errors

8.3 User Acceptance Testing (UAT)

User Acceptance Testing is conducted to evaluate the system from the user's perspective.

- Users tested the system by giving voice commands
- Verified ease of use and response accuracy
- Checked whether the system is user-friendly and understandable
- Feedback confirmed that the system is simple and effective

8.4 Output Testing

Output testing ensures that the system produces correct and expected results.

- Verified correct responses for commands like:
 - “Open Google”
 - “What is the time?”
 - “Search Wikipedia”
- Checked accuracy of information retrieved from external sources
- Ensured proper voice output using text-to-speech

8.5 Performance Testing

Performance testing evaluates system speed and responsiveness.

- Measured response time for voice commands
- Ensured quick processing and minimal delay
- Verified system works smoothly without lag

8.6 Error Handling Testing

Error handling ensures the system behaves properly in unexpected situations.

- Tested unclear or noisy voice input
- Checked system response when command is not recognized
- Verified that system prompts user to repeat command
- Ensured no system crash occurs during errors

CHAPTER 9

RESULTS & OUTPUT

The Jarvis voice assistant was successfully developed and tested. The system is capable of understanding user voice commands and performing the corresponding actions accurately. The results demonstrate that the assistant works efficiently and provides quick responses to user queries.

9.1 Voice Input Recognition

The system successfully captures user voice input through the microphone and converts it into text using the SpeechRecognition library. The accuracy of recognition is high when the input is clear.

9.2 Opening Applications and Websites

Jarvis can open various websites and applications based on user commands.

- Example:
 - “Open Google” → Opens Google website
 - “Open YouTube” → Opens YouTube

The system performs these actions quickly without delay.

9.3 Wikipedia Search

The assistant can fetch information from Wikipedia and provide a brief summary.

- Example:
 - “Search Python in Wikipedia”
- Output:
 - Jarvis reads out a short description of Python

This feature helps users quickly get information without manual searching.

9.4 Music Playback

Jarvis can access local directories and play music files.

- The system lists available songs
- Plays selected music file automatically

This demonstrates the system's ability to interact with local files.

9.5 Time Announcement

The assistant provides real-time system information such as current time.

- Example:
 - “What is the time?”
- Output:
 - Jarvis responds with the current time

9.6 Email Sending

The system includes an email feature that allows users to send emails using voice commands.

- Takes voice input for message content
- Sends email using SMTP protocol

9.7 Voice Response Output

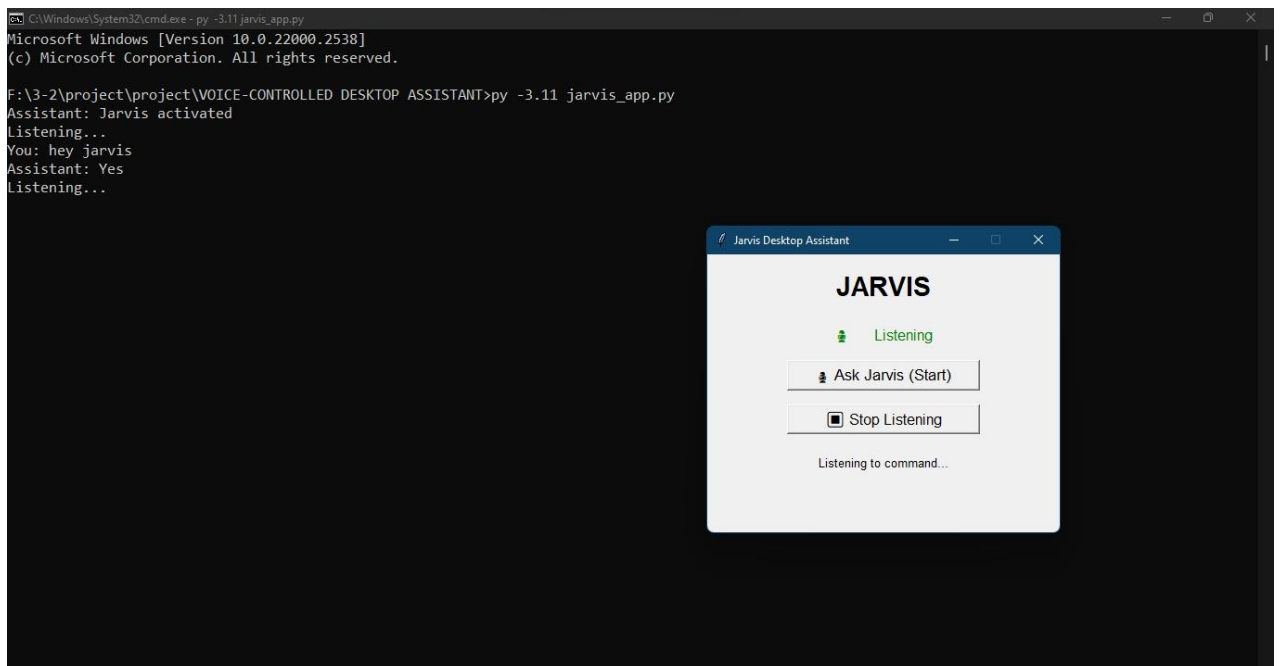
All outputs are converted into speech using the pyttsx3 library.

- The assistant speaks responses clearly
- Provides natural interaction experience

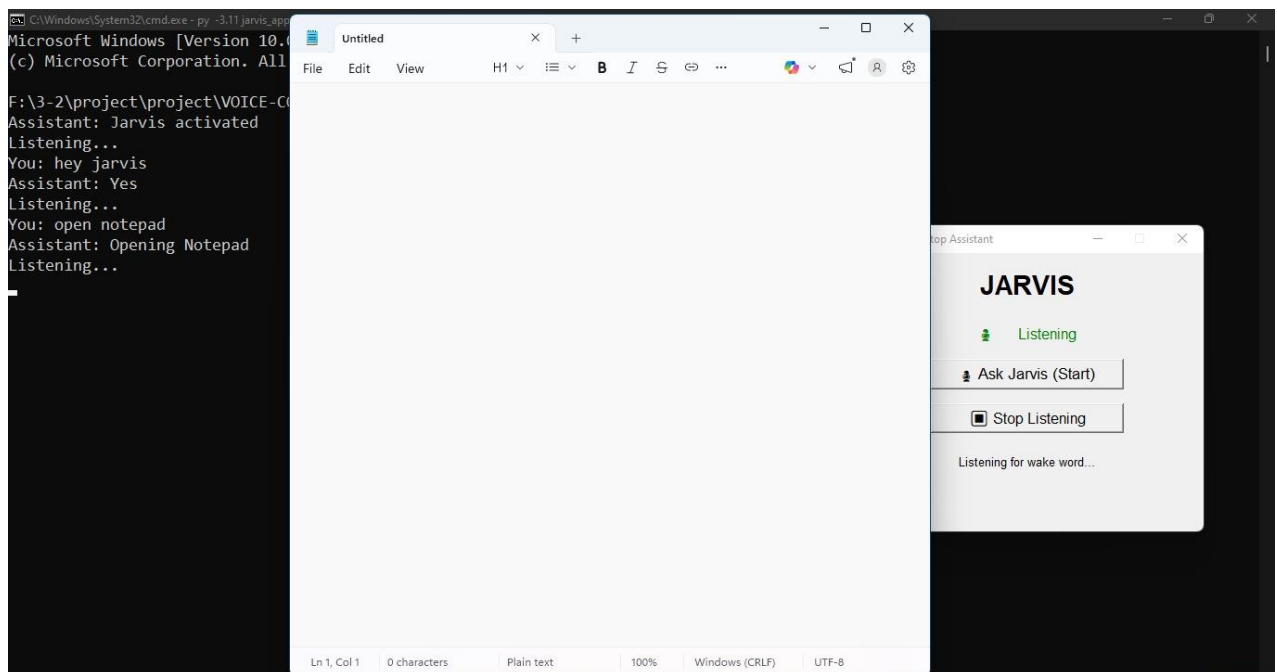
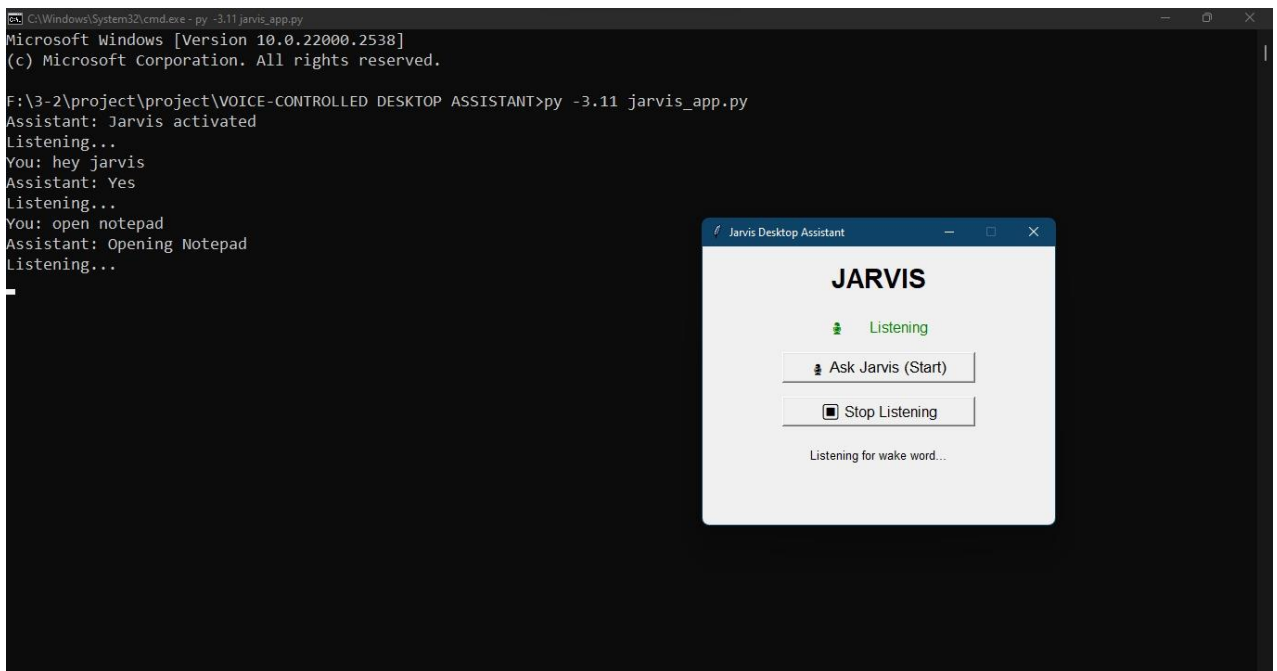
9.8 Overall Performance

- The system responds quickly to commands
- Provides accurate results for predefined tasks
- Works efficiently with minimal errors
- User-friendly and easy to operate

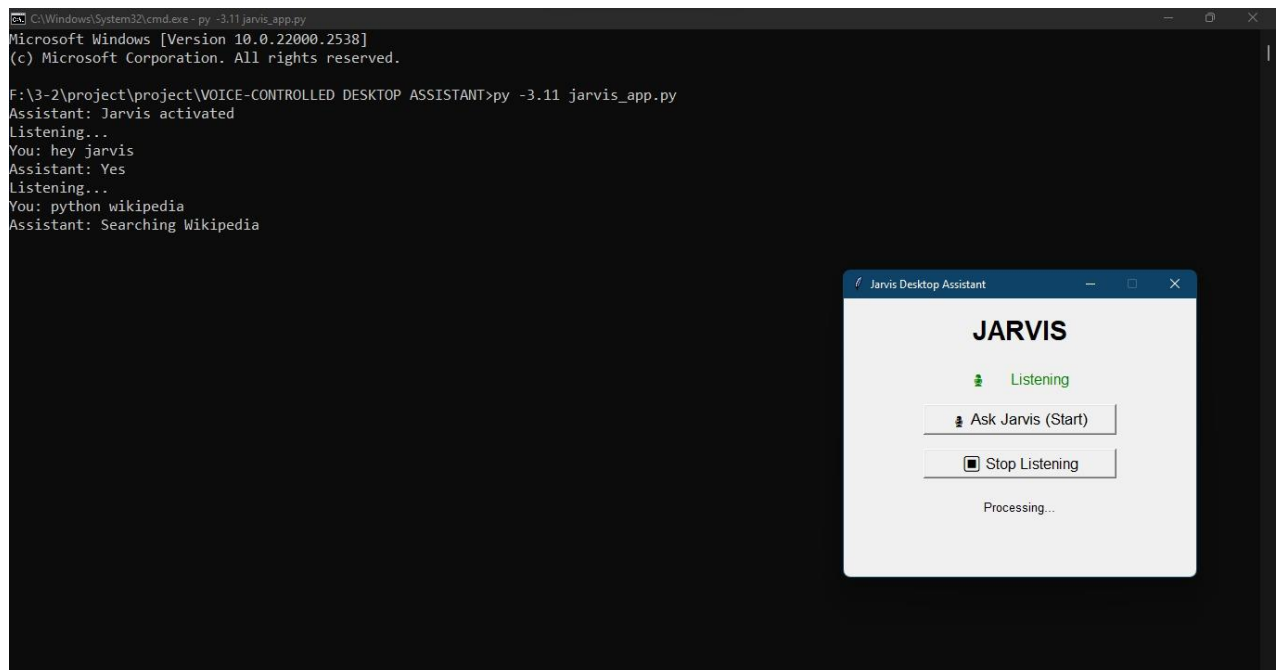
9.9 Sample Output Screens



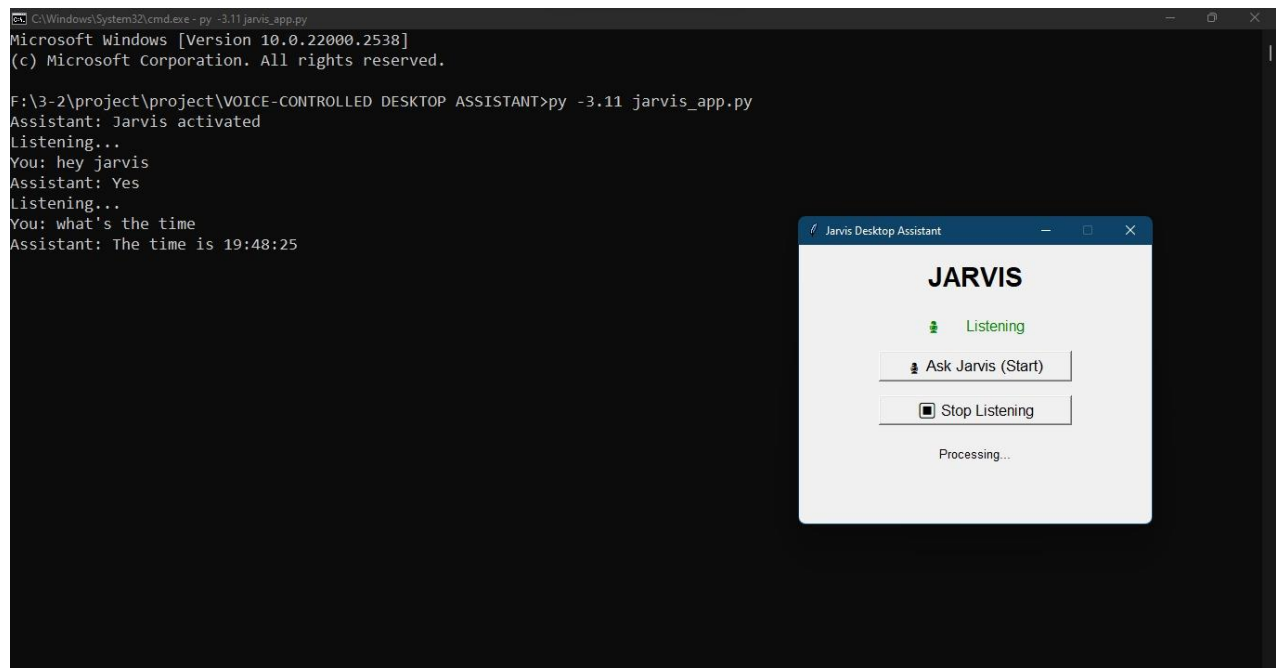
9.9.1 Voice input screen



9.9.2 Command execution output



9.9.3 Wikipedia results



9.9.4 Time Result

CHAPTER 10

CONCLUSION

The Jarvis voice assistant project demonstrates how a simple yet effective system can be developed using Python to enable voice-based interaction with computers. The system successfully captures user voice input, processes it using speech recognition techniques, and performs the required actions such as opening applications, searching information, and providing real-time responses.

The project highlights the importance of automation and user-friendly interfaces in modern computing. By integrating libraries such as SpeechRecognition and pyttsx3, the system provides a smooth and natural interaction experience. It reduces the need for manual input and makes computer usage faster and more convenient.

Overall, the Jarvis voice assistant is a beginner-friendly project that provides practical knowledge of speech processing and automation. It can be further enhanced with advanced features such as artificial intelligence, natural language processing, and integration with smart devices, making it more powerful and efficient in real-world applications.

CHAPTER 11

FUTURE SCOPE

The Jarvis voice assistant developed in this project provides basic functionality for voice-based interaction. However, there is significant scope for further enhancement to make the system more intelligent, efficient, and feature-rich.

11.1 Integration with Artificial Intelligence

The system can be enhanced by integrating Artificial Intelligence and Natural Language Processing (NLP) techniques. This will enable the assistant to understand complex user queries and provide more accurate and meaningful responses.

11.2 Multi-Language Support

Currently, the system primarily supports English commands. In the future, it can be extended to support multiple languages such as Hindi, Telugu, and other regional languages, making it more accessible to a wider range of users.

11.3 Mobile Application Development

The project can be expanded into a mobile application for Android and iOS platforms. This will allow users to access the voice assistant anytime and anywhere.

11.4 Integration with Smart Devices (IoT)

Jarvis can be integrated with smart home devices such as lights, fans, and other appliances. This will enable users to control devices using voice commands, making the system more useful in real-life scenarios.

11.5 Enhanced Security Features

Future versions can include secure authentication methods such as voice recognition or user login systems to ensure privacy and security of user data.

11.6 Advanced Features and Automation

Additional features such as setting reminders, alarms, calendar management, weather updates, and email automation can be added. This will make the assistant more powerful and closer to real-world applications like Siri and Alexa.

CHAPTER 12

REFERENCES

- [1] Python Software Foundation, “Python Documentation,” *Python.org*. [Online]. Available: <https://www.python.org/documentation/>. Accessed: Jan. 2026.
- [2] SpeechRecognition Developers, “SpeechRecognition Library,” *Python Package Index (PyPI)*. [Online]. Available: <https://pypi.org/project/SpeechRecognition/>. Accessed: Jan. 2026.
- [3] pyttsx3 Developers, “pyttsx3: Text-to-Speech Conversion Library,” *Python Package Index (PyPI)*. [Online]. Available: <https://pypi.org/project/pyttsx3/>. Accessed: Jan. 2026.
- [4] Wikimedia Foundation, “MediaWiki API Documentation,” *Wikipedia*. [Online]. Available: https://www.mediawiki.org/wiki/API:Main_page. Accessed: Jan. 2026.
- [5] Google, “Speech-to-Text API Overview,” *Google Cloud*. [Online]. Available: <https://cloud.google.com/speech-to-text>. Accessed: Jan. 2026.
- [6] Tutorials and resources from online programming platforms and developer communities related to Python and voice assistant development.